

Documentación técnica — E-Service (C# / Avalonia)

Este documento explica **a fondo cómo funciona el código** de la reescritura de E-Service, archivo por archivo, y **cómo se conectan entre sí las partes visuales** (`.axaml`) **con la lógica** (`.cs`). Está pensado para que puedas leerlo y entender exactamente qué hace cada pieza, sin necesidad de adivinar nada al abrir el proyecto en VS Code.

1. La idea central: MVVM

Todo el proyecto sigue un patrón llamado **MVVM** (Model-View-ViewModel). Es una evolución directa de la separación que ya tenías en el proyecto original entre `clases` (datos) y `controladores` (lógica), con una capa adicional en medio:

Diagrama de la arquitectura MVVM:

- `Model` → los datos puros (Empleado, Plato, Insumo...)
- `Service` → la lógica de negocio y el guardado en archivos .txt (esto reemplaza tus "controladores")
- `ViewModel` → NUEVO — conecta una pantalla con los Services que necesita
- `View (.axaml)` → lo que se ve en pantalla (botones, textos, imágenes)

Regla de oro del proyecto: el archivo `.axaml` (la View) **nunca** contiene lógica. Solo describe *cómo se ve* la pantalla. Toda decisión ("¿qué pasa cuando hago clic aquí?", "¿qué texto debo mostrar?") vive en el archivo `.cs` del ViewModel correspondiente. Esto es exactamente lo mismo que ya intuías en Java al separar `clases` de `controladores`, solo que aquí se lleva un paso más lejos.

2. Cómo arranca la aplicación

`Program.cs`

Es el equivalente exacto de tu antiguo `Main.java` con el `public static void main`. Su único trabajo es arrancar Avalonia:

```
csharp
public static void Main(string[] args)
=> BuildAvaloniaApp()
    .StartWithClassicDesktopLifetime(args);
```

`StartWithClassicDesktopLifetime` es lo que le dice a Avalonia "esto es una aplicación de escritorio normal, con una ventana, no una app móvil ni web".

`App.axaml` y `App.axaml.cs`

`App.axaml` es donde se registran **dos cosas globales** que usa toda la aplicación:

```
<?xml
<Application.DataTemplates>
  <local:ViewLocator />
</Application.DataTemplates>

<Application.Styles>
  <FluentTheme />
  <StyleInclude Source="avares://EService/Styles.axaml" />
</Application.Styles>
```

`ViewLocator`: explico exactamente qué hace en la sección 3.

`FluentTheme` + `Styles.axaml`: la apariencia visual completa de la app (colores, botones, tarjetas). Se explica en la sección 8.

`App.axaml.cs` tiene el método `OnFrameworkInitializationCompleted()`, que crea **la única ventana que existe en toda la aplicación**:

```
csharp
desktop.MainWindow = new MainWindow
{
    DataContext = new MainWindowViewModel()
};
```

Aquí pasa algo importante: se le asigna un `DataContext`. En Avalonia (y en WPF), el `DataContext` de un control es **el objeto del que ese control lee sus datos mediante bindings**. Al ponerle `MainWindowViewModel` como `DataContext` a la ventana, cualquier `{Binding Xxx}` dentro de esa ventana (o de lo que esa ventana contenga) busca la propiedad `Xxx` dentro de `MainWindowViewModel`.

3. El sistema de navegación (reemplaza tus `new JFrame().setVisible(true)`)

Este es probablemente el cambio conceptual más grande respecto al proyecto en Java, así que lo explico con

detalle.

El problema que resuelve

En tu versión original, cada pantalla era una ventana (`JFrame`) distinta. Para "navegar" a otra pantalla, literalmente creabas una ventana nueva:

```
java
new Order_menu().setVisible(true);
this.dispose(); // cerrar la ventana actual
```

Esto significa que técnicamente tenías **muchas ventanas independientes** abriéndose y cerrándose todo el tiempo.

Cómo funciona ahora

Aquí solo existe **una ventana** (`MainWindow`). "Navegar" a otra pantalla significa simplemente decirle a esa ventana "ahora muestra este otro contenido", sin abrir nada nuevo.

Tres piezas trabajan juntas para lograr esto:

a) `MainWindow.axaml` — la ventana solo tiene un `ContentControl`:

```
xml
<ContentControl Content="{Binding CurrentPage}" />
```

Esto dice: "muestra aquí lo que sea que tenga la propiedad `CurrentPage`".

b) `MainWindowViewModel.cs` — guarda cuál es la pantalla activa:

```
csharp
```

```

public class MainWindowViewModel : ViewModelBase
{
    private ViewModelBase _currentPage;

    public ViewModelBase CurrentPage
    {
        get => _currentPage;
        set => SetField(ref _currentPage, value);
    }

    public MainWindowViewModel()
    {
        NavigationService.Main = this;
        _currentPage = new LoginViewModel(); // pantalla inicial
    }
}

```

Fíjate: `CurrentPage` es de tipo `ViewModelBase` — no es una View, es un **ViewModel**. Aquí es donde entra la tercera pieza.

c) `ViewLocator.cs` — traduce automáticamente "qué ViewModel es este" a "qué View (XAML) debo dibujar":

```

csharp
public Control Build(object? data)
{
    var name = data.GetType().FullName!
        .Replace("ViewModels", "Views")
        .Replace("ViewModel", "View");

    var type = Type.GetType(name);
    return (Control)Activator.CreateInstance(type)!;
}

```

Esto toma el nombre completo de la clase, por ejemplo `EService.ViewModels.OrderMenuViewModel`, y lo convierte en `EService.Views.OrderMenuView` mediante un simple reemplazo de texto. Luego crea una instancia de esa clase (la View) y la muestra.

Por qué importa esto: mientras cada `XxxViewModel` tenga su `XxxView` correspondiente con el mismo nombre (quitando "Model"), **nunca tienes que registrar manualmente las pantallas en ningún lado**. Si algún día agregas una novena pantalla, solo necesitas crear `NuevaPantallaViewModel.cs` + `NuevaPantallaView.axaml` siguiendo el mismo patrón de nombres, y el sistema la reconoce automáticamente.

NavigationService.cs — el botón de "cambiar de pantalla"

```
csharp
public static void NavigateTo(ViewModelBase viewModel)
{
    Main.CurrentPage = viewModel;
}
```

Esta es la línea que reemplaza directamente tu antiguo `new Order_menu().setVisible(true)`. Por ejemplo, en `LoginViewModel.cs`, después de validar el código:

```
csharp
if (empleado.EsGerente)
    NavigationService.NavigateTo(new ManagerDashboardViewModel());
else
    NavigationService.NavigateTo(new OrderMenuViewModel());
```

Al llamar `NavigateTo(new OrderMenuViewModel())`:

Se crea una nueva instancia de `OrderMenuViewModel` (esto ejecuta su constructor, que carga los platos del menú).

Se la asigna a `MainWindowViewModel.CurrentPage`.

Como `CurrentPage` usa `SetField(...)` (heredado de `ViewModelBase`), esto dispara automáticamente el evento `PropertyChanged`.

El `ContentControl` de `MainWindow.axaml`, que está escuchando ese evento por el binding `{Binding CurrentPage}`, se entera del cambio.

El `ContentControl` le pide al `ViewLocator` "¿qué View le corresponde a este `OrderMenuViewModel`?", y el `ViewLocator` responde con una nueva instancia de `OrderMenuView`.

Esa View se dibuja dentro del `ContentControl`, reemplazando lo que había antes.

Todo esto ocurre en el mismo instante, sin abrir ninguna ventana nueva.

4. ViewModelBase.cs y RelayCommand.cs — la "maquinaria" que todo ViewModel usa

ViewModelBase.cs — cómo una pantalla se actualiza sola

Cada ViewModel hereda de esta clase base. Su función es avisarle a la pantalla "oye, uno de mis datos cambió, por favor redibújate":

```
csharp
protected bool SetField<T>(ref T field, T value, [CallerMemberName] string? propertyName = null)
{
    if (Equals(field, value)) return false;
    field = value;
    OnPropertyChanged(propertyName);
    return true;
}
```

Ejemplo de uso real, en `OrderMenuViewModel.cs`:

```
csharp
private int _cantidadEnCarrito;
public int CantidadEnCarrito
{
    get => _cantidadEnCarrito;
    set => SetField(ref _cantidadEnCarrito, value);
}
```

Cuando en algún lado del código se hace `CantidadEnCarrito = 5;`, este `set` llama a `SetField`, que:

Guarda el nuevo valor en el campo privado `_cantidadEnCarrito`.

Dispara `PropertyChanged`.

Cualquier `{Binding CantidadEnCarrito}` en el `.axaml` (como el número dentro del carrito en `OrderMenuView.axaml`) se actualiza en pantalla automáticamente, sin que tengas que escribir ningún código que "busque la etiqueta y le cambie el texto" (como sí hacías en Swing con `label.setText(...)`).

`RelayCommand.cs` — el reemplazo de `ActionListener`

En Java, un botón necesitaba esto:

```
java
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent evt) { ... }
});
```

Aquí, un botón en el `.axaml` simplemente dice:

```
xml
```

```
<Button Content="Ingresar" Command="{Binding LoginCommand}" />
```

Y el ViewModel expone una propiedad de tipo `ICommand`:

```
csharp
public ICommand LoginCommand { get; }

public LoginViewModel()
{
    LoginCommand = new RelayCommand(Login);
}

private async void Login() { ... }
```

`RelayCommand` es una clase pequeña, escrita a mano en este proyecto (para no depender de librerías externas adicionales), que simplemente envuelve un método normal de C# para que Avalonia sepa "esto se puede ejecutar cuando el usuario interactúa con este control". Existe también `RelayCommand<T>`, usado cuando el comando necesita un dato extra — por ejemplo, en `OrderMenuViewModel.cs`:

```
csharp
AgregarAlCarritoCommand = new RelayCommand<MenuItemViewModel>(AgregarAlCarrito);
```

En este caso, cada botón de plato en el menú pasa **cuál plato específico** fue presionado, mediante

`CommandParameter="{Binding}"` en el XAML (ver sección 6.3).

5. Capa de Modelos (`Models/`)

Son clases de datos puros, sin ninguna lógica de negocio — igual que en tu proyecto original, solo que usando propiedades de C# (`{ get; set; }`) en vez de campos públicos de Java.

Archivo	Qué representa
<code>Empleado.cs</code>	Un usuario del sistema: <code>Codigo</code> + <code>EsGerente</code> (booleano que decide a qué pantalla va después de iniciar sesión)
<code>Insumo.cs</code>	Un ingrediente en inventario: nombre, unidad, cantidad, código
<code>Plato.cs</code>	Un ítem del menú: nombre, tipo, precio, código, y su receta (<code>InsumosUsados[]</code> + <code>CantidadesConsumidas[]</code> , dos arreglos paralelos que dicen qué ingredientes consume y cuánto de cada uno)
<code>Transaction.cs</code>	Un movimiento financiero (venta o compra), con su propio ID único generado con <code>DateTimeOffset.UtcNow.ToUnixTimeMilliseconds()</code>
<code>Cliente.cs</code> , <code>Pedido.cs</code>	Se mantienen del proyecto original para uso futuro, tal como estaban sin usarse activamente en las pantallas

Un detalle importante está en `Plato.cs` — la propiedad calculada `IconFileName`, que traduce un código como `"B013"` al nombre real del archivo de imagen `"b13.png"`:



```
csharp
public string IconFileName
{
    get
    {
        var letter = char.ToLowerInvariant(Codigo[0]);
        var number = Codigo[1..];
        var twoDigit = int.TryParse(number, out var n) ? n.ToString("00") : "00";
        return $"{letter}{twoDigit}.png";
    }
}
```

Esto existe porque en tus recursos originales, los íconos usan 2 dígitos (`b13.png`) mientras que los códigos de plato usan 3 (`B013`) — esta propiedad hace ese "puente" una sola vez, en un solo lugar.

6. Capa de Servicios (`Services/`) — donde vive toda la lógica de negocio

6.1 FileDataStore.cs — reemplaza FileManager.java

Hace exactamente el mismo trabajo que tenías antes: leer y escribir archivos `.txt` separados por `|`. Lo único que cambió es **dónde** se guardan los archivos:

```
csharp
private static readonly string BaseDirectory =
    Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData), "EService");
```

Tu versión original tenía escrito literalmente `"C:\EserviceData\..."`, que solo funciona en Windows. `Environment.SpecialFolder.ApplicationData` le pregunta al sistema operativo "¿cuál es la carpeta de datos de aplicaciones del usuario?", y responde correctamente sin importar si el programa corre en Linux, Windows o Mac. El **formato interno** de los archivos (`|` como separador) no cambió en absoluto.

6.2 LoginService.cs — reemplaza LoginController.java

Guarda los empleados en un `Dictionary<string, Empleado>` (antes probablemente un array o lista en tu versión). Los métodos son prácticamente idénticos en comportamiento:

```
csharp
public void AgregarEmpleado(string codigo, bool esGerente)
{
    if (_empleados.ContainsKey(codigo))
        throw new InvalidOperationException($"El código {codigo} ya existe en el sistema.");

    _empleados[codigo] = new Empleado(codigo, esGerente);
    GuardarEmpleados();
}
```

Diferencia clave respecto al original: en Java, este controlador llamaba directamente a `JOptionPane.showMessageDialog(...)` para avisar errores. Aquí, en cambio, el método **lanza una excepción** (`throw new InvalidOperationException(...)`) y es el ViewModel quien decide cómo mostrar ese error (ver sección 7). Esto separa "la lógica de qué está mal" de "cómo se lo muestro al usuario" — permite, por ejemplo, cambiar el diseño de los mensajes de error sin tocar esta clase.

6.3 InsumoService.cs — reemplaza InsumoController.java

Contiene el catálogo por defecto de ~74 ingredientes (idéntico al original) y la lógica de sumar/restar

cantidades:

```

csharp
public void AgregarInsumo(Insumo insumo)
{
    var existente = BuscarInsumoPorCodigo(insumo.Codigo);
    if (existente != null)
        existente.Cantidad += insumo.Cantidad; // "top up" del stock existente
    else
        _insumos.Add(insumo);
    GuardarInsumos();
}

```

Un cambio pequeño pero real: el original usaba un arreglo de tamaño fijo (`Insumo[100]`), lo que ponía un límite de 100 insumos. Aquí se usa `List<Insumo>`, que crece automáticamente — el límite artificial desaparece sin que se haya cambiado ninguna otra lógica.

6.4 `PlatoService.cs` — reemplaza `PlatoController.java`

Contiene los 40 platos del menú (10 platos fuertes, 5 entradas, 6 postres, 19 bebidas), cada uno con su receta exacta copiada del original. Por ejemplo, la Mojarra frita:

```

csharp
Add("Mojarra frita", "Plato fuerte", 25000, "P000",
    new[] { "I000", "I002", "I012", "I007", "I009", "I008", "I004", "I005", "I040", "I010" },
    new[] { 1, 170, 28, 0.10, 1, 0.25, 0.50, 0.40, 0.50, 20.0 });

```

El método `PorCategoria(string categoria)` es lo que permite que `OrderMenuViewModel` pida "dame solo los platos fuertes" sin tener que recorrer manualmente los 40 ítems cada vez.

6.5 `TransactionService.cs` — reemplaza `TransactionController.java`

Mantiene el libro de ventas/compras y los totales acumulados (`TotalSales`, `TotalPurchases`, `NetProfit`), leídos y guardados con el mismo formato `|` de siempre. La diferencia respecto al original: ya no construye una tabla de Swing (`DefaultTableModel`) aquí — eso es trabajo de la View, no del Service (ver `TransactionRowViewModel` en la sección 7.5).

6.6 `CartService.cs` — reemplaza `Carrito.java`

Prácticamente idéntico al original: una lista de códigos de plato que se va llenando mientras el mesero toma el pedido, y se vacía al confirmar.

6.7 `DialogService.cs` — NUEVO, reemplaza `JOptionPane`

Avalonia no tiene un equivalente directo a `JOptionPane`. Este servicio resuelve eso ofreciendo 4 métodos simples que cualquier ViewModel puede llamar:

```
csharp
public static Task ShowMessageAsync(string title, string message);
public static Task<bool> ShowConfirmAsync(string title, string message);
public static Task<string?> ShowInputAsync(string title, string message);
public static Task<string?> ShowChoiceAsync(string title, string message, string[] options);
```

Cada uno abre una ventanita pequeña (definida en `Views/Dialogs/`, ver sección 9) y **espera** (`await`) a que el usuario responda antes de continuar — esto imita exactamente el comportamiento "bloqueante" que tenía `JOptionPane` en Swing, pero sin congelar toda la aplicación mientras espera (gracias a `async`/`await`).

Ejemplo real de uso, en `OrderMenuViewModel.cs`:

```
csharp
var cantidadStr = await DialogService.ShowInputAsync(
    "Agregar al carrito",
    $"Producto: {plato.Nombre}\n...");

if (cantidadStr is null) return; // el usuario canceló
```

6.8 `AssetImageLoader.cs` — NUEVO, carga los íconos de los platos

```
csharp
public static Bitmap? TryLoad(string relativePath)
{
    var uri = new System.Uri($"avares://{EService}/{relativePath}");
    using var stream = AssetLoader.Open(uri);
    return new Bitmap(stream);
}
```

Los 40 íconos de platos (copiados de tu carpeta `Recursos/` original) están "incrustados" dentro del propio programa (gracias a la línea `<AvaloniaResource Include="Assets\*" />` en `EService.csproj`). Este método los busca por su ruta y los convierte en algo que un control `<Image>` puede mostrar. Si un ícono no existe, devuelve `null` en vez de hacer fallar toda la pantalla — así un ícono faltante no rompe el menú completo.

7. Capa de ViewModels — el "cerebro" de cada pantalla

Aquí es donde vive la lógica específica de cada pantalla: qué datos mostrar, qué pasa al presionar cada botón.

7.1 LoginViewModel.cs (pantalla: LoginView.axaml)

```
csharp
private async void Login()
{
    var empleado = LoginService.Instance.ValidarCodigo(Codigo);

    if (empleado is null)
    {
        await DialogService.ShowMessageAsync("Error", "Código inválido");
        return;
    }

    LoginService.Instance.CurrentUserCode = Codigo;
    LoginService.Instance.CurrentUserIsManager = empleado.EsGerente;

    if (empleado.EsGerente)
        NavigationService.NavigateTo(new ManagerDashboardViewModel());
    else
        NavigationService.NavigateTo(new OrderMenuViewModel());
}
```

Esto es la traducción directa de tu antiguo `botondeingresoActionPerformed(...)` de `Beginningpage.java`:
valida el código, y según el rol, navega a una pantalla u otra.

7.2 OrderMenuViewModel.cs (pantalla: OrderMenuView.axaml)

Es el reemplazo de `Order_menu.java` (tu archivo más grande, con 895 líneas). En vez de 40 botones ubicados manualmente uno por uno, este ViewModel expone **listas** que la View recorre automáticamente:

```
csharp
public List<MenuItemViewModel> PlatosFuentes { get; }
public List<MenuItemViewModel> Entradas { get; }
public List<MenuItemViewModel> Postres { get; }
public List<MenuItemViewModel> Bebidas { get; }
```

Cada `MenuItemViewModel` envuelve un `Plato` y le agrega lo que la pantalla necesita mostrar (precio ya formateado, ícono ya cargado):

csharp

```
public class MenuItemViewModel
{
    public Plato Plato { get; }
    public string Nombre => Plato.Nombre;
    public string PrecioTexto => $"${Plato.Precio N0}";
    public Bitmap? Icono { get; }
}
```

El flujo de "agregar al carrito" (`AgregarAlCarrito`) hace, en orden, exactamente lo mismo que la versión Java:

csharp

```
private async void AgregarAlCarrito(MenuItemViewModel? item)
{
    var cantidadStr = await DialogService.ShowInputAsync(...); // 1. pedir cantidad
    if (cantidadStr is null) return;

    var confirmar = await DialogService.ShowConfirmAsync(...); // 2. confirmar
    if (!confirmar) return;

    for (var i = 0; i < cantidad; i++)
        _cart.AgregarPlato(plato.Codigo); // 3. agregar al carrito

    CantidadEnCarrito = _cart.CantidadItems; // 4. actualizar el contador visible
    await DialogService.ShowMessageAsync("Éxito", ...); // 5. avisar éxito
}
```

7.3 `CheckoutViewModel.cs` (pantalla: `CheckoutView.axaml`)

Reemplaza `TerminarCompra.java`. Al confirmar el pedido, ejecuta en orden:

```
private async void Confirmar()
{
    DescontarInsumos(); // 1. descuenta inventario
    TransactionService.Instance.AddSale(Total, ...); // 2. registra la venta
    GenerarArchivoPedido(currentUser); // 3. escribe el recibo .txt
    _cart.LimpiarCarrito(); // 4. vacía el carrito
    NavigationService.NavigateTo(new OrderMenuViewModel()); // 5. vuelve al menú
}
```

`DescontarInsumos()` recorre cada plato del carrito y, usando los arreglos

`InsumosUsados` / `CantidadesConsumidas` del `Plato` (ver sección 5), resta exactamente esa cantidad de cada ingrediente en `InsumoService` — la misma lógica de descuento de inventario que tenía el original, sin cambios.

7.4 `ManagerDashboardViewModel.cs` (pantalla: `ManagerDashboardView.axaml`)

Reemplaza `manager01page.java`. Su trabajo principal es **traducir datos crudos en algo listo para mostrar**:

```
csharp
Transacciones = _transactionService.Transactions
    .OrderByDescending(t => t.Timestamp)
    .Select(t => new TransactionRowViewModel(t))
    .ToList();
```

7.5 `TransactionRowViewModel.cs` — el reemplazo de tu `TransactionCellRenderer`

En Java, tenías una clase especial (`TransactionCellRenderer`) que pintaba cada fila de la tabla de un color según si era venta o compra. Aquí, ese color se calcula **una sola vez**, al crear cada fila:

```
csharp
RowBackground = t.Type switch
{
    "COMPRA" => new SolidColorBrush(Color.Parse("#FFE0E0")), // rojo suave
    "VENTA" => new SolidColorBrush(Color.Parse("#E1F5E1")), // verde suave
    _ => Brushes.Transparent
};
```

Y en `ManagerDashboardView.axaml`, cada fila simplemente se pinta con ese color ya calculado:

```
xml
<Border Background="{Binding RowBackground}" ...>
```

7.6 InventoryViewModel.cs, AddPurchaseViewModel.cs, UserManagementViewModel.cs

Reemplazan `Inventario.java`, `manageragregarcompra.java` y `usuariosManagerPage.java` respectivamente. Siguen el mismo patrón que ya se explicó: cargan datos de un Service, los envuelven en un "RowViewModel" listo para mostrar, y exponen `ICommand`s para cada botón. Vale la pena mencionar `AddPurchaseViewModel.cs`, que junta **tres pasos del original en una sola confirmación**:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Xamarin.Forms;
using Xamarin.Forms.Xaml;
using EService;
using EService.Models;
using EService.Views;
using EService.ViewModels;

namespace EService.ViewModels
{
    public class InventoryViewModel : BaseViewModel
    {
        private readonly IInsumoService _insumoService;
        private readonly ITransactionService _transactionService;
        private readonly IFacturaService _facturaService;

        public InventoryViewModel(IInsumoService insumoService, ITransactionService transactionService, IFacturaService facturaService)
        {
            _insumoService = insumoService;
            _transactionService = transactionService;
            _facturaService = facturaService;
        }

        public async Task<List<Linea>> AgregarInsumo(Insumo insumo)
        {
            // 1. sube el stock
            await _insumoService.AgregarInsumo(insumo);

            // 2. registra la compra
            await _transactionService.AddPurchase(CostoTotal, ...);

            // 3. escribe la factura .txt
            await _facturaService.GenerarFactura(invoiceNumber, currentUser);
        }
    }
}
```

8. Capa de Views (.axaml) — cómo se conectan con su ViewModel

Cada `View` declara, en su primera línea, **con qué tipo de ViewModel va a trabajar**, usando `x:DataType`:

```
<UserControl ...
    xmlns:vm="using:EService.ViewModels"
    x:Class="EService.Views.LoginView"
    x:DataType="vm:LoginViewModel">
```

Esto le dice al compilador de XAML: "todo `{Binding Xxx}` que escriba dentro de este archivo se refiere a una propiedad de `LoginViewModel`". Gracias a esto, si escribes mal el nombre de una propiedad en el XAML (por ejemplo `{Binding Codgio}` en vez de `{Binding Codigo}`), **el proyecto no compila** — te avisa el error en el momento de compilar, en vez de fallar silenciosamente en tiempo de ejecución como sí podía pasar en Swing con errores de UI mal detectados.

8.1 Casos especiales: `x:CompileBindings="False"`

En `OrderMenuView.axaml`, `ManagerDashboardView.axaml`, `AddPurchaseView.axaml` y `UserManagementView.axaml`, vas a notar esta línea extra en la etiqueta raíz:

```
<UserControl ...
    x:CompileBindings="False">
```

Esto ocurre porque estas pantallas tienen **listas dentro de listas**: por ejemplo, en el menú de pedidos, cada tarjeta de plato (un `MenuItemViewModel`) necesita ejecutar un comando que en realidad vive en la pantalla completa (`OrderMenuViewModel.AgregarAlCarritoCommand`), no en el plato mismo. Para poder "saltar" desde el contexto de un plato individual hasta el comando de la pantalla completa, se usa un tipo de binding más flexible (por nombre de elemento), en vez del binding estrictamente tipado:

```
<Button Command="{Binding DataContext.AgregarAlCarritoCommand, ElementName=RootControl}"
  CommandParameter="{Binding}">
```

Aquí, `ElementName=RootControl` busca el control raíz de la pantalla (que tiene `x:Name="RootControl"`), toma su `DataContext` (que es el `OrderMenuViewModel` completo), y de ahí saca `AgregarAlCarritoCommand`. `CommandParameter="{Binding}"` (sin nombre de propiedad) significa "pásale el objeto completo del que se trata esta tarjeta" — es decir, el `MenuItemViewModel` específico en el que se hizo clic.

8.2 Ejemplo completo de conexión: el botón de un plato

Repasemos, de punta a punta, qué pasa cuando alguien hace clic en la tarjeta de "Mojarra frita" en el menú:

XAML (`OrderMenuView.axaml`): el botón tiene `Command="{Binding DataContext.AgregarAlCarritoCommand, ElementName=RootControl}"` y `CommandParameter="{Binding}"`.

Al hacer clic, Avalonia ejecuta ese comando, pasándole como parámetro el `MenuItemViewModel` de la Mojarra frita.

C# (`OrderMenuViewModel.cs`): el comando fue creado así: `new RelayCommand<MenuItemViewModel>(AgregarAlCarrito)` — así que ejecutar el comando llama al método `AgregarAlCarrito(item)`.

`AgregarAlCarrito` pide la cantidad (`DialogService.ShowInputAsync`), confirma (`DialogService.ShowConfirmAsync`), y si todo sale bien, llama a `_cart.AgregarPlato(plato.Codigo)` tantas veces como la cantidad indicada.

Actualiza `CantidadEnCarrito`, lo cual (gracias a `SetField` en `ViewModelBase`) dispara `PropertyChanged`.

De vuelta al XAML: el contador del carrito, enlazado con `{Binding CantidadEnCarrito}`, se actualiza solo en pantalla.

Ningún paso de este flujo requiere que el `.axaml` "sepa" cómo funciona la lógica — solo declara *qué* debe pasar (ejecutar tal comando con tal parámetro), y el ViewModel decide *cómo*.

9. Los diálogos (`Views/Dialogs/`)

Como se explicó en 6.7, Avalonia no trae `JOptionPane`. Se construyeron 4 ventanas reutilizables:

Archivo	Equivalente a	Devuelve
<code>MessageDialogWindow.axaml</code>	<code>JOptionPane.showMessageDialog</code>	nada (solo un botón "Aceptar")
<code>ConfirmDialogWindow.axaml</code>	<code>JOptionPane.showConfirmDialog</code>	<code>bool</code> (sí/no)
<code>InputDialogWindow.axaml</code>	<code>JOptionPane.showInputDialog</code>	<code>string?</code> (texto o <code>null</code> si canceló)
<code>ChoiceDialogWindow.axaml</code>	<code>JOptionPane.showOptionDialog</code>	<code>string?</code> (opción elegida)

Cada una sigue el mismo patrón: es una ventana pequeña (`Window`, no `UserControl`, porque necesita aparecer flotando encima de la ventana principal) con un método estático `ShowAsync(...)` que la abre y espera la respuesta:

```

csharp
public static Task<bool> ShowAsync(Window owner, string title, string message)
{
    var dialog = new ConfirmDialogWindow(title, message);
    return dialog.ShowDialog<bool>(owner);
}

```

`owner` es la ventana principal — se configura una sola vez, en `MainWindow.axaml.cs`:

```

csharp
public MainWindow()
{
    InitializeComponent();
    DialogService.OwnerWindow = this;
}

```

Así, cualquier ViewModel puede simplemente escribir `await DialogService.ShowConfirmAsync(...)` sin preocuparse por cuál ventana debe ser el "dueño" del diálogo — eso ya está resuelto de antemano.

10. `Styles.axaml` — de dónde salen los colores y el diseño

Este archivo centraliza **toda** la apariencia visual, para que cambiar un color en un solo lugar actualice toda la aplicación. Dos partes:

a) Paleta de colores (arriba del archivo):



```
xml
<Color x:Key="ColorPrimary">#2E7D6B</Color>
<SolidColorBrush x:Key="BrushPrimary" Color="{StaticResource ColorPrimary}" />
```

b) Clases de estilo reutilizables, aplicadas mediante `Classes="..."` en cualquier control:



```
xml
<Style Selector="Button.primary">
  <Setter Property="Background" Value="{StaticResource BrushPrimary}" />
  ...
</Style>
```

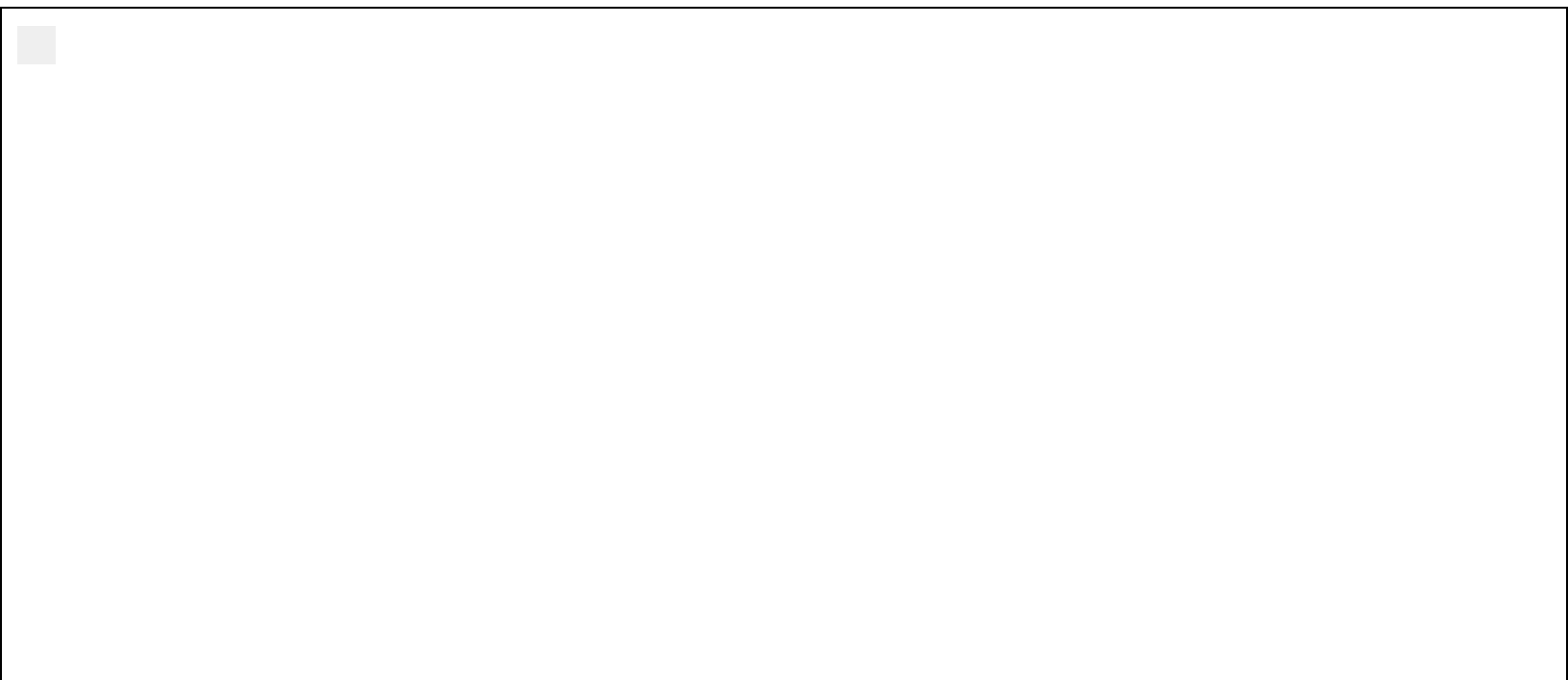
Y en cualquier View:



```
xml
<Button Content="Ingresar" Classes="primary" ... />
```

Esto es el equivalente a tener una "hoja de estilos" central, similar en espíritu a un archivo CSS: en vez de configurar el color de cada botón uno por uno en cada pantalla, defines la clase `primary` una sola vez, y la reutilizas en las 8 pantallas.

11. Mapa rápido de todas las conexiones



Program.cs

- └─ arranca → App.axaml.cs
 - └─ crea → MainWindow (DataContext = MainWindowViewModel)
 - └─ contiene → ContentControl bound a CurrentPage
 - └─ ViewLocator convierte CurrentPage (un ViewModel) en la View correspondiente

Cada pantalla:

XxxView.axaml ← x:DataType → XxxViewModel.cs

Binding (leer datos)	Services.Instance (leer/guardar datos)
Command (ejecutar acciones)	DialogService (mostrar popups)
	NavigationService (cambiar de pantalla)
▼	▼
se ve en pantalla	FileDataStore.cs → archivos .txt en disco

12. Si quieres seguir modificando el proyecto

¿Quieres agregar un plato nuevo al menú? Solo edita `Services/PlatoService.cs` y agrega una línea `Add(...)` — aparece automáticamente en el menú, sin tocar ningún `.axaml`.

¿Quieres cambiar un color? Edita `Styles.axaml`, no busques en cada View individual.

¿Quieres agregar una pantalla nueva? Crea `Services/` (si necesita lógica nueva) +

`ViewModels/NuevaViewModel.cs`

`Views/NuevaView.axaml`, siguiendo el mismo patrón de nombres que las 8 pantallas existentes — el

`ViewLocator` la reconocerá sola.

¿Quieres cambiar qué pasa al hacer clic en un botón? Busca el `ICommand` correspondiente en el ViewModel de esa pantalla, no en el `.axaml`.